

Funciones para manejar memoria dinámica en c++ y c

-**malloc(size)**

Reserva un bloque de memoria del tamaño indicado en bytes.

La memoria no se inicializa y devuelve un puntero al inicio del bloque.

-**realloc(ptr, new_size)**

Cambia el tamaño de un bloque de memoria ya reservado.

Puede mover el bloque a otra zona si no hay espacio continuo.

-**free(ptr)**

Libera la memoria previamente reservada.

Después de usarla, el puntero deja de ser válido.

C++ (3 mecanismos)

-**new**

Reserva memoria dinámicamente y llama al constructor del objeto.

Devuelve un puntero tipado.

-**delete**

Libera memoria reservada con new y llama al destructor.

Es obligatorio para evitar fugas de memoria.

-**std::vector**

Contenedor dinámico que gestiona su propia memoria.

Crece o se reduce automáticamente según se agregan o eliminan elementos.



Funciones para manejar memoria dinámica en python

- **list**

Arreglo dinámico que puede crecer o disminuir en tiempo de ejecución.
La memoria se ajusta automáticamente.

- **dict**

Estructura clave-valor dinámica.

Permite agregar y eliminar pares sin definir tamaño previo.

- **Objetos (class)**

Se crean dinámicamente en memoria.

Python gestiona su ciclo de vida mediante el recolector de basura.



CUADRO COMPARATIVO

		
Ventajas	Sintaxis simple y legible, facilita el aprendizaje y desarrollo rápido. Manejo automático de memoria (menos errores por el programador). Gran cantidad de librerías para ciencia de datos, IA y automatización.	Alto rendimiento y ejecución muy rápida. Control total de la memoria y recursos del sistema. Muy usado en sistemas operativos, drivers y sistemas embebidos.
Desventajas	Menor rendimiento comparado con C. Menos control sobre el hardware y la memoria. No es ideal para sistemas embebidos o de bajo nivel.	Sintaxis más compleja y propensa a errores. Manejo manual de memoria (riesgo de fugas y fallos). Desarrollo más lento y mantenimiento más difícil.
Conclusiones	• Lenguaje de alto nivel orientado a productividad. • Ideal para análisis de datos, IA y automatización. • Reduce errores comunes de memoria. • Sacrifica rendimiento por facilidad de uso.	• Lenguaje de bajo nivel enfocado en eficiencia. • Fundamental para entender cómo funciona el sistema. • Ideal para software crítico y embebido. • Requiere mayor disciplina del programador.